

DAM: Deep Reinforcement Learning based Preload Algorithm with Action Masking for Short Video Streaming

Si-Ze Qian

Communication University of China
qiansize@cuc.edu.cn

Yuhong Xie

Communication University of China
xieyh@cuc.edu.cn

Zipeng Pan

Communication University of China
cuc_pzp@cuc.edu.cn

Yuan Zhang

State Key Laboratory of Media
Convergence and Communication,
Communication University of China
yzhang@cuc.edu.cn

Tao Lin*

State Key Laboratory of Media
Convergence and Communication,
Communication University of China
lintao@cuc.edu.cn

ABSTRACT

Short video streaming has been increasingly popular in recent years. Due to its unique user behavior of watching and sliding, a critical technique issue is to design a preload algorithm deciding which video chunk to download next, bitrate selection and the pause time, in order to improve user experience while reducing bandwidth wastage. However, designing such a preload algorithm is non-trivial, especially taking into account conflicting goals of improving QoE and reducing bandwidth wastage. In this paper, we propose a deep reinforcement learning-based approach to simultaneously decide the aforementioned three decision variables via learning an optimal policy under a complex environment of varying network conditions and unpredictable user behavior. In particular, we incorporate domain knowledge into the decision procedure via action masking to make decisions more transparent, and accelerate the model training. Experimental results validate the proposed approach significantly outperforms baseline algorithms in terms of QoE metrics and bandwidth wastage.

CCS CONCEPTS

• Information systems → Multimedia streaming.

KEYWORDS

Short video streaming, reinforcement learning, action masking

ACM Reference Format:

Si-Ze Qian, Yuhong Xie, Zipeng Pan, Yuan Zhang, and Tao Lin. 2022. DAM: Deep Reinforcement Learning based Preload Algorithm with Action Masking for Short Video Streaming. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, October 10–14, 2022, Lisboa, Portugal. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3503161.3551573>

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '22, October 10–14, 2022, Lisboa, Portugal

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9203-7/22/10...\$15.00

<https://doi.org/10.1145/3503161.3551573>

1 INTRODUCTION

Short video streaming has been tremendously popular with the emerging and development of commercial APPs like TikTok[9]. Different from traditional long videos, short video applications enable users to view the videos in a manner of "watching and sliding", which means users can slide away from the current video at any time and watch the next one in the recommended video queue. In order to ensure user experience, prefetching these videos at the client-side buffer is a prerequisite. However, once a user slides to the next video, the remaining downloaded chunks of the previous one will become useless and result in bandwidth wastage. Therefore, on one hand, a preload algorithm of short video streaming must be elaborately designed to decide which video chunk to download next and its corresponding bitrate. On the other hand, considering the huge bandwidth cost of short video platforms, it also needs to decide the pause time during which the download procedure pauses to reduce the bandwidth wastage.

However, designing such a preload algorithm for short video streaming is non-trivial. First, deciding the video chunk to download, bitrate selection, and the pause time highly depends on varying network conditions and user watching behavior both of which are difficult to predict. Second, it is challenging to model the complex interactions among the three decision variables, varying network conditions, unpredictable user behavior and QoE(Quality of Experience) metrics such as rebuffering events and video quality. Last but not least, the conflicting goals make the problem more challenging, i.e., improving user QoE indicates prefetching as many video chunks as possible, while saving bandwidth requires to limit bandwidth usage.

The problem of short video preloading is far from well investigated. APL[17] is one of the few schemes to solve this problem via only choosing download video chunk and pause time, while ignoring bitrate selection. Extensive research, including rule-based [2, 7, 12, 15, 16] and learning-based [3, 5, 6, 10, 18], have been conducted on ABR (Adaptive Bitrate Streaming) problem of traditional videos. However, they cannot be directly applied to the scenario of short video since they just address the problem of bitrate selection.

In this paper, we design a deep RL(Deep Reinforcement Learning) based framework, DAM for short, to simultaneously decide the video chunk to download, bitrate selection and the pause time via learning an optimal policy under a complex environment of varying network conditions and unpredictable user behavior. Furthermore,

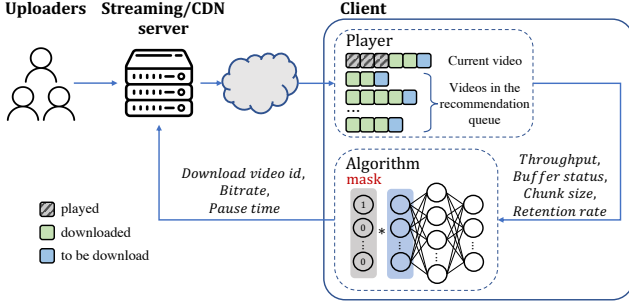


Figure 1: Short Video Streaming System Overview

we incorporate domain knowledge into RL-based decision-making via action masking[4] to make the decisions more transparent[13], while accelerating the learning procedure. Experimental results validate that the proposed approach significantly outperforms the baseline methods in terms of an overall score representing both QoE metrics and bandwidth wastage. Meanwhile, our proposed method is also proved to achieve faster convergence within 30 iterations, thanks to the well-dedicated design of the framework and introduced domain knowledge via action masking.

2 PROBLEM DEFINITION

Figure 1 depicts a system overview of short video streaming. User generated short videos are uploaded to the streaming server. These videos are divided into sequential chunks with a fixed duration and then each of chunk is encoded into multiple bitrates. At the client side, except for the currently playing video, some extra (up to 4) videos are placed in the recommendation queue. Thus, users can slide away anytime and continue to watch the next video smoothly. Considering dynamic network condition and unpredictable user behaviors, the client needs a preload algorithm to implement a smart video prefetching policy.

The preload problem of short video streaming is formally defined as follows: given the inputs of chunk download throughput, buffer status, size of chunks to be downloaded and retention rates, the preload algorithm outputs video ID to download, corresponding bitrate, and the pause time(in the unit of milliseconds). It is worth noting that the retention rate of a short video refers to the probability that users are still watching some chunk of this video, which is obtained by collecting a large number of user behaviors on the streaming/CDN(Content Delivery Network) server.

3 PROPOSED METHOD

In this section, we present the design and implementation of our approach. Figure 1 illustrates how the algorithm interacts with a short video player. We begin with an introduction of the proposed RL model, followed by action masking to incorporate with domain knowledge to make decision-making more transparent and reasonable. Finally, we presents the implementation in detail.

3.1 Deep Reinforcement Learning Framework for Short Video Streaming

DAM utilizes deep reinforcement learning to generate a mapping from a set of states to an optimal decision. As shown in Figure 2,

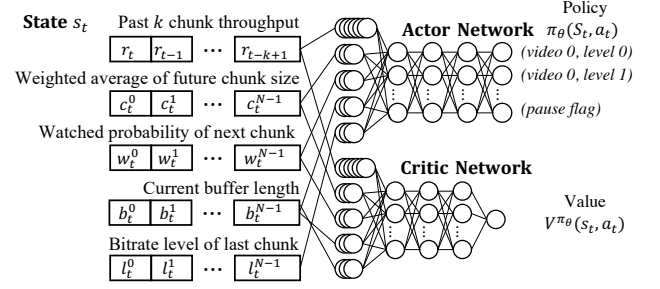


Figure 2: Model Architecture

DAM is trained with PPO[14], i.e., an actor-critic RL algorithm that has the advantages of simplicity and effectiveness.

State: At each step of decision-making, the agent takes state S_t as the inputs of neural networks. S_t is defined as $S_t = \{\vec{r}_t, \vec{c}_t^n, \vec{w}_t^n, \vec{b}_t^n, \vec{l}_t^n\}$ where $\vec{r}_t$ is the network throughput for the past k chunks. $\vec{c}_t^n$ is the weighted average size of the next three chunks for up to N videos, including the currently playing video and the remaining ones in the recommended queue. The weighting factor for the future three chunks is set to $[0.7, 0.2, 0.1]$. $\vec{w}_t^n$ is the conditional probability that the user watches the next chunk of each video. This value is calculated by dividing the retention rate of the next chunk of each video by the retention rate of the currently playing chunk of that video. $\vec{b}_t^n$ stands for the current buffer length of each video. And $\vec{l}_t^n$ is defined as the bitrate level of last chunk for each of these videos.

Action: In this task, the agent outputs three decisions, namely, the video chunk to download, the bitrate and the pause time. However, the action of pause is different from other actions. If the agent decides to pause for a while, it means that none of chunks will be selected for download. For this reason, the first 15 actions are generated via combining 5 videos with 3 bitrate levels. The actions ranging from (video 0, level 0) to (video 4, level 2) mark the chunk and bitrate level that the algorithm chooses to download. Note that once the video ID to download is decided by the agent, our algorithm will choose to download the next chunk of this video by default. Then we add a pause flag as the last action, as shown in Figure 2. Once the agent selects this action, the client pauses downloading for a fixed period. Since the agent can choose the pause action repeatedly to extend the pause period, a short pause time allows the agent to fine-tune the pause period. However, this setting is not conducive to training because the state only changes slightly after a short pause. To balance the utility and precision of pausing, we set a pause time of 250ms for the pause flag.

Reward: After applying an action to download a chunk or pause a while, the environment returns a reward R . A basic design of R is provided by ACM Multimedia 2022 Grand Challenge[1] as follows:

$$R = u_i \cdot (\text{bitrate}_i - \text{smooth}_i) - 1.85 \cdot \text{rebuff}_i - 0.5 \cdot \text{band_usage}_i \quad (1)$$

$$u_i = \begin{cases} 0, & \text{user will not watch chunk } i \\ 1, & \text{user will watch chunk } i \end{cases} \quad (2)$$

where u_i is determined by the user's behavior. bitrate_i is a fixed value of $[0.75, 1.2, 1.85]$ depending on the bitrate level of the chunk i , and smooth_i is defined as the difference of the bitrate between the

Algorithm 1: Action Masking

Input : Number of videos N , Action probability $p[3N + 1]$,
Current buffer length $b[N]$, Number of remain
chunk $r[N]$

Output: New action probability $p_{\text{new}}[3N + 1]$

```

1 Initialize mask  $m[3N + 1]$  with 0;
2 for  $n \leftarrow 0$  to  $N - 1$  do
3   if  $n=0$  and  $r[n]>0$  then
4     /* video 0 is the playing video */
5      $m[3n : 3n + 3] \leftarrow 1$ ;
6     if  $b[n]<1.0$  then
7       break;
8     end
9      $m[3N] \leftarrow 1$ ;
10  end
11  if  $0 < n < N$  and  $r[n] > 0$  then
12    if  $b[n] \leq 3.0$  and  $b[n - 1] > 0$  then
13       $m[3n : 3n + 3] \leftarrow 1$ ;
14    end
15  end
16  $\vec{p}_{\text{new}} \leftarrow \vec{p} * \vec{m}$ ;
```

chunk i and the previous chunk. $rebuff_i$ and $band_usage_i$ represent the rebuffering time and the bandwidth usage for downloading chunk i , respectively.

Different from the basic design of reward function, we design a modified one using w_i as a weight of a part of reward metrics:

$$R_{\text{mod}} = u_i \cdot w_i \cdot (\text{bitrate}_i - \text{smooth}_i) - 1.85 \cdot rebuff_i - 0.5 \cdot band_usage_i \quad (3)$$

where w_i is the probability that the user will watch chunk i . The motivation for using the modified reward function stems from a critical observation. That is, from the perspective of statistics, the probability of a user viewing a specific chunk will increase as the number of chunks of the video being viewed increases. For example, only a few users watching the second chunk will eventually watch the ninth chunk, while most users who have watched the eighth chunk will continue to watch the ninth chunk. Using the modified reward function, downloading a chunk with a lower probability of viewing will get a lower reward, which encourages the agent to choose the pause flag until the probability becomes higher. As a result, it helps to reduce bandwidth waste. The effectiveness of the modified reward will be discussed in Section 4.3.

3.2 Action Masking

Aiming to make decision-making more transparent and accelerate learning progress[13], we incorporate domain knowledge into policy learning. Specifically, we introduce a masking vector $\vec{m}$ into DAM by multiplying $\vec{m}$ with the action probability $\vec{p}$ to calculate the new probability distribution $\vec{p}_{\text{new}}$, as described in Algorithm 1. The elements of $\vec{m}$ will be determined depending on whether the corresponding action is reasonable. 0 means the action is not available and cannot be chosen, while 1 means normal. Based on the analysis of this task, we set up the following hand-crafted rules:

- Completely downloaded videos cannot be preloaded again.
- Currently playing video should be downloaded immediately once its playback buffer length is less than one second.
- All videos need to be downloaded in order, i.e., the agent cannot preload video n until the video $n - 1$ has started to be cached.
- The buffer of each video in the recommendation queue should not exceed three seconds for the purpose of bandwidth saving.

Note that even with the aforementioned rules, DAM is an end-to-end trained system. That is, end-to-end training of such a system attempts to mitigate the sub-optimal problem via training a model with different components jointly.

3.3 Implementation

Here we present the network settings and other implementation details. In our settings, the number of videos N is set to 5 and we consider past $k = 10$ chunks as input. Our RL agent is based on the actor-critic framework, composed of an actor-network and a critic-network. The actor-network adopts a simple fully connected (FC) layer, outputting the vector of probabilities for each action. Specifically, inputs are passed to 3 hidden layers with 128, 64, and 64 neurons respectively, using ReLU function. And the last output layer has 16 neurons applying a Sigmoid function. The critic-network employs a similar network structure, estimating the value of a state. The difference is that the critic network only has 1 neuron in the last layer without any activation function.

During training, we use the Adam[8] optimizer to update network parameters with initial learning rates of $3 * 10^{-4}$ and $3 * 10^{-3}$ for the actor and critic, respectively.

4 EXPERIMENTS

4.1 Simulator and Datasets

We evaluate our algorithm in the simulator provided by ACM MM'22 Grand Challenge[1]. The simulator can automatically generate user's viewing behavior according to user retention rate and simulate a short video player under different network conditions. Then, considering both QoE and bandwidth usage, the simulator will give a score to reflect the algorithm's performance, which refers to the accumulation of R for each step. The datasets used in this paper consist of network traces and video traces:

Network: There are three categories of network traces including "low", "medium" and "high". Each of them represents network conditions of poor, medium and good, respectively. Each trace records the timestamp and average bandwidth at a granularity of 0.5 second.

Video: We expand the official dataset from 7 videos to 55 videos to fully test the impact of different video types on the algorithm performance. The meta data of each video consists of the "short_video_size" recording the size of chunks with different bitrates, and the corresponding "user_ret" describing user retention ratio of each chunk. Each video is encoded into three target bitrates in the range of [750, 1200, 1850] kbps.

4.2 Algorithms for Comparison

We implement our proposed approach as well as two baseline algorithms: (1) **RobustMPC**[16]: A default algorithm which is provided

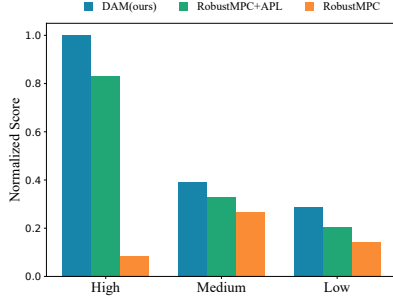


Figure 3: Comparison of Average Score under Different Network Conditions

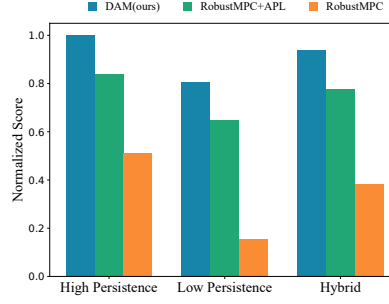


Figure 4: Comparison of Average Score of Different Video Categories

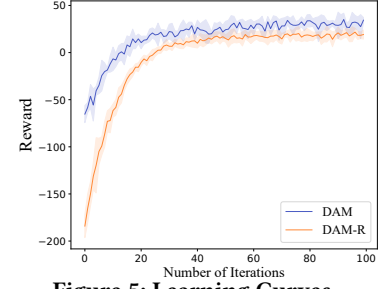


Figure 5: Learning Curves

Table 1: Detailed Performance Comparison of DAM with its Variants

Model	Total score	Total QoE	QoE factors			Bandwidth usage	Wasted ratio
			Quality	Rebuff	Smooth		
DAM-none	64.83	242.55	302.91	-27.12	-33.21	177.72	18.70%
DAM-M	67.16	247.27	312.16	-33.94	-30.94	180.11	17.36%
DAM-R	71.54	238.45	298.81	-28.29	-32.06	166.91	15.80%
DAM	79.65	246.72	306.42	-30.71	-28.98	167.07	14.53%

by the competition. RobustMPC is a solution for ABR problem of traditional videos. Here it is used in a manner of just deciding bitrate level and downloading videos sequentially, while ignoring the problem of video chunk selection and pause time. (2) **RobustMPC+APL**: An algorithm that uses RobustMPC for deciding bitrate and uses APL[17] for chunk selection and pause-time decision. APL formulates the preload problem into a user experience optimization problem to maximize the playback smoothness and minimize the bandwidth waste. Then APL solves this problem with a single-step greedy algorithm using Lyapunov optimization theory[11].

4.3 Experiment Analysis

Overall Performance. To evaluate the performance of our proposed method, we compare it with RobustMPC and RobustMPC+APL under a variety of network conditions and video traces. Figure 3 shows that DAM achieves the best performance under all network conditions. Compared with RobustMPC+APL, DAM improves the normalized score by 19%-41%, which validates its high network adaptability. Further, we analyze the algorithm performance under different video viewing durations. We divide all videos into high persistence and low persistence, depending on whether more than 50% of users who watch the video slide away within 10 seconds. Figure 4 illustrates that DAM can also achieve the best performance by obtaining 19%-24% gain in different video sets.

Effectiveness of Action Masking and Modified Reward

Function. In order to evaluate the effectiveness of action masking and modified reward R_{mod} , we trained three models for comparison: (1) DAM-none, a model using R without action masking. (2) DAM-M, a model using R with action masking. (3) DAM-R, a model using R_{mod} without action masking. DAM is evaluated in detail against its variants in terms of bitrate utility, rebuffering penalty, smoothness penalty, bandwidth usage penalty, and bandwidth wasted ratio. As shown in Table 1, DAM improves the total QoE by 3.4% compared

with DAM-R, and DAM-M improves the total QoE by 2% compared with DAM-none, both of which validate that action masking with domain knowledge does rationalize decision making. Additionally, DAM reduces bandwidth waste by 16.3% compared with DAM-M, and DAM-R reduces bandwidth waste by 15.5% compared with DAM-none, demonstrating the effectiveness of R_{mod} in reducing bandwidth waste. Furthermore, it shows that DAM significantly improves the total score by 23.1%, compared with DAM-none where neither modified reward nor action masking is adopted.

Accelerating Learning via Action Masking. Finally, we conduct multiple training sessions with DAM and DAM-R to investigate the effectiveness of action masking on accelerating training procedure. Figure 5 depicts the learning curves of DAM and DAM-R in training process where the shaded area represents the standard deviation of reward over random seeds, and the solid line shows the mean value. Experiment shows that DAM needs 30 iterations to reach convergence, while DAM-R needs 40 iterations on average, with a 33% improvement in convergence speed.

5 CONCLUSION

In this paper, we propose a novel preload approach named DAM for short video streaming, which employs a RL-based agent to decide the video chunk to download, its bitrate and the pause time simultaneously. Along with the RL-based model, DAM utilizes an action masking mechanism to leverage domain knowledge in order to make decision-making more reasonable and transparent. Experiments demonstrate that DAM significantly outperforms other baselines under various network conditions and video traces.

ACKNOWLEDGEMENT

This work is supported by the National Key Research and Development Program of China (No. 2021YFF0900503) and the National Natural Science Foundation of China under Grant 61971382.

REFERENCES

- [1] 2022. *AltransCompetition/Short-Video-Streaming-Challenge: ACM MULTIMEDIA 2022 Grand Challenge*. Retrieved June 9, 2022 from <https://github.com/AltransCompetition/Short-Video-Streaming-Challenge>
- [2] Zahaib Akhtar, Yun Seong Nam, Ramesh Govindan, Sanjay Rao, Jessica Chen, Ethan Katz-Bassett, Bruno Ribeiro, Jibin Zhan, and Hui Zhang. 2018. Oboe: Auto-tuning video ABR algorithms to network conditions. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 44–58.
- [3] Matteo Gadaleta, Federico Chiariotti, Michele Rossi, and Andrea Zanella. 2017. D-DASH: A deep Q-learning framework for DASH video streaming. *IEEE Transactions on Cognitive Communications and Networking* 3, 4 (2017), 703–718.
- [4] Shengyi Huang and Santiago Ontañón. 2020. A closer look at invalid action masking in policy gradient algorithms. *arXiv preprint arXiv:2006.14171* (2020).
- [5] Tianchi Huang, Rui-Xiao Zhang, Chao Zhou, and Lifeng Sun. 2018. Qarc: Video quality aware rate control for real-time video streaming based on deep reinforcement learning. In *Proceedings of the 26th ACM international conference on Multimedia*. 1208–1216.
- [6] Tianchi Huang, Chao Zhou, Rui-Xiao Zhang, Chenglei Wu, Xin Yao, and Lifeng Sun. 2019. Comyco: Quality-aware adaptive video streaming via imitation learning. In *Proceedings of the 27th ACM International Conference on Multimedia*. 429–437.
- [7] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. 2014. A buffer-based approach to rate adaptation: Evidence from a large video streaming service. In *Proceedings of the 2014 ACM conference on SIGCOMM*. 187–198.
- [8] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [9] Qian Li, Yuan Zhang, Hong Huang, and Jinyao Yan. 2019. Deep learning-based short video recommendation and prefetching for mobile commuting users. In *Proceedings of the ACM SIGCOMM 2019 Workshop on Networking for Emerging Applications and Technologies*. 49–55.
- [10] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural adaptive video streaming with pensieve. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 197–210.
- [11] Michael J Neely. 2010. Stochastic network optimization with application to communication and queueing systems. *Synthesis Lectures on Communication Networks* 3, 1 (2010), 1–211.
- [12] Yanyuan Qin, Chinmaey Shende, Cheonjin Park, Subhabrata Sen, and Bing Wang. 2021. DataPlanner: data-budget driven approach to resource-efficient ABR streaming. In *Proceedings of the 12th ACM Multimedia Systems Conference*. 94–107.
- [13] Rajkumar Ramamurthy, Christian Bauckhage, Rafet Sifa, Jannis Schücker, and Stefan Wrobel. 2019. Leveraging domain knowledge for reinforcement learning using MMC architectures. In *International Conference on Artificial Neural Networks*. Springer, 595–607.
- [14] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [15] Kevin Spiteri, Rahul Uргаonkar, and Ramesh K Sitaraman. 2020. BOLA: Near-optimal bitrate adaptation for online videos. *IEEE/ACM Transactions on Networking* 28, 4 (2020), 1698–1711.
- [16] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. 2015. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 325–338.
- [17] Haodan Zhang, Yixuan Ban, Xinggong Zhang, Zongming Guo, Zhimin Xu, Shengbin Meng, Junlin Li, and Yue Wang. 2020. APL: Adaptive Preloading of Short Video with Lyapunov Optimization. In *2020 IEEE International Conference on Visual Communications and Image Processing (VCIP)*. IEEE, 13–16.
- [18] Huanhuan Zhang, Anfu Zhou, Jiamin Lu, Ruoxuan Ma, Yuhua Hu, Cong Li, Xinyu Zhang, Huadong Ma, and Xiaojiang Chen. 2020. OnRL: improving mobile video telephony via online reinforcement learning. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. 1–14.